

# 編譯器製作 HW2

陳宥辰 B112040016

## 1. 版本

版本：flex 2.6.4

版本：bison (GNU Bison) 3.5.1

編譯器：gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0

## 2. 作業平台

VirtualBox 虛擬機運行 Ubuntu Linux。

## 3. 執行方式

使用Ubuntu的終端機進行編譯以及測試：

```
> make all           (編譯出執行檔)
> ./calc < filename.pas (執行)
> make clean        (清除檔案)
```

## 4. 如何處理規格書上的問題

### (1). 未宣告變數：

建立符號表，當解析到變數階段時會呼叫declare\_temp\_ids(\$3) 將變數名稱與對應的型態（整數=1，實數=2，陣列=3，字串=4）存入symtable 中。當解析到變數被使用時，會查詢 symtable。若找不到該變數會回傳 -1，程式會立刻印出 Line X: '變數名' is not defined，同時將屬性 \$\$ = -1 往上傳遞，標記這是一個無效變數，並設定 has\_error = 1。

### (2). 結構缺失：

根據 Bison 內建的錯誤偵測及定義函式 yyerror()，當缺少特定關鍵字，例如 if 的後方缺少 then，Bison 會觸發 syntax error，yyerror()則會印出錯誤訊息。

### (3). 型態不匹配

利用 Yacc 的 %union 定義了 type\_val。每個變數 (varid) 或常數 (INT\_NUM, STRING\_LITERAL 等) 在被解析時，都會把自己的型態代碼賦值給 \$\$。

在 simpexp (例如加減法) 與 assign (賦值) 規則中，會去檢查左右兩邊子節點的型態 (例如 \$1 和 \$3)。若發現 \$1 != \$3 就會呼叫 type\_name() 函數將代碼轉回字串，並印出如 Type mismatch: cannot add integer and string 的錯誤，抓到型態不同的錯誤。

#### (4). 缺少符號或使用錯誤符號：

除了 Bison 抓出符號遺漏之外，將容易混淆的符號在文法中加入特別規則。離如，為了處理錯誤將 `‘:=’` 寫成 `‘=’`，在文法中加入 `‘assign_eq: ‘=’`，當配對到錯誤符號時主動報錯並進行錯誤恢復，以確保解析器能繼續執行。若程式結尾缺少 `‘.’`，需要在 `‘prog’` 中加入了對應的規則來捕捉並提示 `‘end of input found’`。

#### (5). 錯誤恢復：

在 `‘stmt_list’` 與 `‘dec_list’` 等核心列表規則中，加入了 `‘| error stmt_sep’` 的機制。當發生語法錯誤時，Parser 會自動丟棄無法辨識的 token，直到遇到同步符號（分號 `‘;’`），並重置錯誤狀態（`‘has_error = 0; yyerrok;’`），繼續正確解析下一行程式碼。

### 5. 遇到的問題與解決方案

問題：語法錯誤恢復導致 Yacc 進入無限迴圈

說明：在實作錯誤恢復時，為了讓 Parser 遇到錯誤不會中斷檢查，最初在 `stmt` 規則中加入了 `error { yyerrok; }`。後來發現這會導致 Parser 宣告錯誤已修復，卻不會強制消耗錯誤的 token。下一輪解析時，Parser 再次讀到同一個錯誤的 token，再次宣告修復，導致進入迴圈，直接就卡住了。

解決：改變錯誤恢復的同步策略。將 `yyerrok` 從中途或單一 `stmt` 規則中分開，改到包含分隔符號（如分號 `;` 或 `end`）的列表遞迴規則（`stmt_list` 與 `body`）中。讓程式碼在錯誤時會把錯誤 token 消去，直到遇到分號或區塊結束，才執行 `yyerrok`，避免進入無限迴圈。

問題：Lexer 與 Parser 的錯誤標記狀態同步不一致

說明：為了阻止發生錯誤的原始碼被印出來，Lexer 使用 `has_error` 標記。但如果直接在 Lexer 換行時將 `has_error` 歸零，當 Yacc 出錯且尚未遇到分號完成同步恢復時，Lexer 會把標記清空。這會導致接下來還在恐慌模式中的被誤判為正確的一行直接印出，造成輸出錯誤。

解決：將 `has_error` 狀態的清空權限給 Yacc 的 `stmt_sep` 與 `body` 的結尾來控制。只要 Yacc 還沒抓到分號，標記就維持為 1。Lexer 僅負責讀取與遵從標記，確保 Lexer 與 Parser 的同步。

問題：宣告錯誤導致後方連續錯誤

說明：如 `error3` 在前方宣告發生語法錯誤，該行宣告會失敗。導致後續程式碼即便正確使用變數，也會因為變數沒進符號表而印出訊息 `‘i’ is not defined’`。

解決：在變數宣告的 `error` 規則中（`‘dec: error’`），若宣告時出錯，還是要呼叫 `‘declare_temp_ids(1)’` 將已經讀取到的變數名稱存為 `‘integer’`。這樣可以讓後方的未宣告訊息去除。

## 6. 所有測試檔執行出來的結果，存成圖片或文字檔

correct.pas:

```
youchen@youchen-VirtualBox:~/Desktop/compiler/HW2$ ./calc < correct.pas
Line 1: program test;
Line 2: var
Line 4:   i, j: integer;
Line 5:   ans: array[0 .. 81] of integer;
Line 6: begin
Line 7:   i := -1+3;
Line 8:   j := +7*8;
Line 9:   ans[0] := 7;
Line 14:   for i:=1 to 9 do
Line 15:   begin
Line 16:     for j:=1 to i do
Line 17:       ans[i*9+j] := i*j;
Line 18:   end;
Line 20:   for i:=1 to 9 do
Line 21:   begin
Line 22:     for j:=1 to i do
Line 23:       if ( ans[i*9+j] mod 2 = 0) then
Line 24:         write(i, '*', j, '=', ans[i*9+j], ' ');
Line 25:       writeln;
Line 26:   end;
Line 27: end.
```

error1.pas:

```
youchen@youchen-VirtualBox:~/Desktop/compiler/HW2$ ./calc < error1.pas
Line 1: program test;
Line 2: var
Line 3:   i: integer;
Line 4: begin
Line 5, at char 5, := expected but = found
Line 6: 'j' is not defined
Line 6, at char 5, := expected but = found
Line 7: 'j' is not defined
Line 8, at char 5, then expected but write found
Line 8:   Write('ok');
Line 9: end.
```

error2.pas:

```
youchen@youchen-VirtualBox:~/Desktop/compiler/HW2$ ./calc < error2.pas
Line 1: program test;
Line 2: var
Line 3:   i, j : integer;
Line 4: begin
Line 5:   i := 5*2;
Line 6:   j := 9;
Line 7:   if (i > j)
Line 8, at char 5, then expected but write found
Line 8:   Write('ok');
Line 9: end.
```

error3.pas:

```
youchen@youchen-VirtualBox:~/Desktop/compiler/HW2$ ./calc < error3.pas
Line 1: program test;
Line 2: var
Line 3, at char 8, : expected but := found
Line 4: begin
Line 5:   i := 5;
Line 6, at char 1, . expected but end found
```

error4.pas:

```
youchen@youchen-VirtualBox:~/Desktop/compiler/HW2$ ./calc < error4.pas
Line 1: program test;
Line 2: var
Line 3:   i, j : integer;
Line 4:   c : string;
Line 5: begin
Line 6:   i := 5;
Line 7:   c := 'aa';
Line 8, at char 5, := expected but = found
Line 8: Type mismatch: cannot add integer and string
Line 9: end.
```